



INSTITUTO POLITECNICO NACIONAL

Escuela Superior de Computo

Arquitectura de las Computadoras

Simulación en RARS

Profesor: Miguel Angel Aleman Arce

Alumno: Gutiérrez Espinosa Jordan Gabriel

Grupo: 5CV1

Ciclo: 2022-2023(1)

En la creación de un programa en RARS se tiene que tener en cuenta los registros que la componen, así como los códigos que la modifican ya que este compilador esta mas orientado a microchips los cuales hay que darles instrucciones mas especificas sobre los movimientos en los registros, como el siguiente programa de un bucle mostrado en la Figura 1.

```

Edit  Execute
contador.asm  contador (1).asm
1  |-- Programa contador
2  |-- El registro x5 se incremeta indefinidamente
3
4      .text
5
6      |-- Inicializar el registro x5 a 0
7      addi x5, x0, 0
8
9  bucle:
10     |-- Incrementar el registro x5 en una unidad
11     addi x5, x5, 1  |-- x5 = x5 + 1
12
13     |-- Repetir indefinidamente
14     b bucle
15

```

Figura 1, Primer ejercicio.

Vemos como inicializa el registro 5, 6 y 7 en distintos valores.

Después añadimos un bucle en el que a x5 se le aumentara 1 por cada vez que el ciclo se repita como se muestra en la Figura 2 Cuando compilamos nos sale esta ventana con las instrucciones a realizar y los resultados del proceso a la derecha, Después de darle 8 veces al botón podemos ver que el registro 5 cambio de valor a 7.

Text Segment				
Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x00000293	addi x5,x0,0x00000000	7: addi x5, x0, 0
☐	0x00400004	0x00128293	addi x5,x5,0x00000001	11: addi x5, x5, 1 -- x5 = x5 + 1
☐	0x00400008	0xffdff06f	jal x0,0xffffffff	14: b bucle

Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x7ffffeffc
gp	3	0x10008000
tp	4	0x00000000
t0	5	0x00000007
t1	6	0x00000000
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x00000000
a1	11	0x00000000
a2	12	0x00000000

Figura 2, ejecución primer Ejercicio.

Compilación programa Hola mundo:

Y en la Figura 3 se muestra como se puede mostrar texto en el RARS con un programa simple de hola mundo.

contador.asm

contador (1).asm

nola_mundo.asm

```

1  |-- Programa hola mundo en RISC-V
2  |-- Esto son comentarios
3      .data
4  str:  .asciz "Hola mundo en RISC-V!\n"
5
6      .text
7
8  main:
9
10     la a0, str
11     li a7, 4
12     ecall
13
14     li a7, 10
15     ecall
16
17
18

```

Edit

Execute

Text Segment

Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x0fc10517	auipc x10,0x0000fc10	10: la a0, str
☐	0x00400004	0x00050513	addi x10,x10,0x0000...	
☐	0x00400008	0x00400893	addi x17,x0,0x00000004	11: li a7, 4
☐	0x0040000c	0x00000073	ecall	12: ecall
☐	0x00400010	0x00a00893	addi x17,x0,0x0000000a	14: li a7, 10
☐	0x00400014	0x00000073	ecall	15: ecall

Messages

Run I/O

Hola mundo en RISC-V!

Hola mundo en RISC-V!

-- program is finished running (0) --

Hola mundo en RISC-V!

-- program is finished running (0) --

Clear

Figura 3, Hola mundo.



Preguntas:

- ¿Qué instrucción es la que hace que aparezca el mensaje de texto en la consola?

☐	0x00400010	0x00a00893	addi x17,x0,0x0000000a	14:	li a7, 10
☐	0x00400014	0x00000073	ecall	15:	ecall

- ¿Qué instrucción es la que hace que el programa termine?

☐	0x00400010	0x00a00893	addi x17,x0,0x0000000a	14:	li a7, 10
☐	0x00400014	0x00000073	ecall	15:	ecall

- ¿Sabes cuál es el código máquina de esta instrucción?

0x00000073

- ¿Cuántos bits tiene esta instrucción?

10

- ¿Cuántas instrucciones se ejecuten hasta que el programa termina?

6

- ¿Cual crees que es la dirección donde está situada la primera instrucción?



Ejercicio1:

```
#-- Programa contador
#-- El registro x5 se incremeta indefinidamente

.text

#-- Inicializar el registro x5 a 0
addi x3, x0, 3
addi x4, x0, 4
addi x5, x0, 5
addi x6, x0, 6
addi x7, x0, 7
addi x8, x0, 8
```

Bkpt	Address	Code	Basic	
☐	0x00400000	0x00300193	addi x3,x0,0x00000003	7: addi x3, x0, 3
☐	0x00400004	0x00400213	addi x4,x0,0x00000004	8: addi x4, x0, 4
☐	0x00400008	0x00500293	addi x5,x0,0x00000005	9: addi x5, x0, 5
☐	0x0040000c	0x00600313	addi x6,x0,0x00000006	10: addi x6, x0, 6
☐	0x00400010	0x00700393	addi x7,x0,0x00000007	11: addi x7, x0, 7
☐	0x00400014	0x00800413	addi x8,x0,0x00000008	12: addi x8, x0, 8

sp	2	0x7ffefffc
gp	3	0x00000003
tp	4	0x00000004
t0	5	0x00000005
t1	6	0x00000006
t2	7	0x00000007
s0	8	0x00000008
s1	9	0x00000000

Ejercicio 2:

Se Hizo lo mismo con el registro 1.

Name	Number	Value
zero	0	0x00000000
ra	1	0x00000002
sp	2	0x00000063
gp	3	0x00000000

Text Segment					Registers		
Bkpt	Address	Code	Basic	Source	Name	Number	Value
☐	0x00400000	0x00000093	addi x1,x0,0x00000000	7: addi x1, x0, 0	zero	0	0x00000000
☐	0x00400004	0x06300113	addi x2,x0,0x00000063	8: addi x2, x0, 99 #-- le agregamos el valor de 9 al registro 2	ra	1	0x00000002
☐	0x00400008	0x00000193	addi x3,x0,0x00000000	9: addi x3, x0, 0	sp	2	0x00000063
☐	0x0040000c	0x00208093	addi x1,x1,0x00000002	13: addi x1, x1, 2 #-- x1 toma el su mismo valor mas 2	gp	3	0x00000000
☐	0x00400010	0xffff006f	jai x0,0xffffffe	16: b bucle	tp	4	0x00000000
					t0	5	0x00000000
					t1	6	0x00000000

ent					Registers		
Bkpt	Address	Code	Basic	Source	Name	Number	Value
00000	0x00000093	addi x1,x0,0x00000000	7: addi x1, x0, 0		zero	0	0x00000000
00004	0x06300113	addi x2,x0,0x00000063	8: addi x2, x0, 99 #-- le agregamos el valor de 9 al registro 2		ra	1	0x00000002
00008	0x00000193	addi x3,x0,0x00000000	9: addi x3, x0, 0		sp	2	0x00000063
0000c	0x00208093	addi x1,x1,0x00000002	13: addi x1, x1, 2 #-- x1 toma el su mismo valor mas 2		gp	3	0x00000000
00010	0xffff006f	jai x0,0xffffffe	16: b bucle		tp	4	0x00000000
					t0	5	0x00000000
					t1	6	0x00000000



Ejercicio 3:

```
ntador.asm | contador(1).asm |
#-- Programa contador
#-- El registro x5 se incremeta indefinidamente

.text

#-- Inicializar el registro x5 a 0
addi x3, x0, 0
addi x4, x0, 0 #-- le agregamos el valor de 9 al registro 2
addi x5, x0, 0

bucle:
#-- Incrementar el registro x5 en una unidad
addi x3, x3, 1 #-- x3 toma el su mismo valor mas 1
addi x4, x4, 3 #-- x4 toma el su mismo valor mas 3
addi x5, x5, 5 #-- x5 toma el su mismo valor mas 5

#-- Repetir indefinidamente
b bucle
```

Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x00000193	addi x3,x0,0x00000000	7: addi x3, x0, 0
☐	0x00400004	0x00000213	addi x4,x0,0x00000000	8: addi x4, x0, 0 #-- le agregamos el valor de 9 al registro 2
☐	0x00400008	0x00000293	addi x5,x0,0x00000000	9: addi x5, x0, 0
☐	0x0040000c	0x00118193	addi x3,x3,0x00000001	13: addi x3, x3, 1 #-- x3 toma el su mismo valor mas 1
☐	0x00400010	0x00320213	addi x4,x4,0x00000003	14: addi x4, x4, 3 #-- x4 toma el su mismo valor mas 3
☐	0x00400014	0x00528293	addi x5,x5,0x00000005	15: addi x5, x5, 5 #-- x5 toma el su mismo valor mas 5
☐	0x00400018	0xff5ff06f	jal x0,0xffffffa	18: b bucle

gp	3	0x00000001
tp	4	0x00000003
t0	5	0x00000005
t1	6	0x00000000
-	-	-
gp	3	0x00000002
tp	4	0x00000006
t0	5	0x0000000a
t1	6	0x00000000
-	-	-
sp	2	0x7fffeffc
gp	3	0x00000007
tp	4	0x00000015
t0	5	0x00000023
t1	6	0x00000000

Ejercicio 4:

```

#-- Programa contador
#-- El registro x5 se incremeta indefinidamente

    .text

#-- Inicializar el registro x5 a 0
addi x6, x0, 0

bucle:
#-- Incrementar el registro x5 en una unidad
addi x6, x6, -1 #-- x6 toma el su mismo valor menos 1

#-- Repetir indefinidamente
b bucle

```

Bkpt	Address	Code	Basic	
☐	0x00400000	0x00000313	addi x6,x0,0x00000000	7: addi x6, x0, 0
☐	0x00400004	0xffff30313	addi x6,x6,0xffffffff	11: addi x6, x6, -1 #-- x6 toma el su mismo valor menos 1
☐	0x00400008	0xffdff06f	jal x0,0xffffffffe	15: b bucle

tp			
t0	4	0x00000000	
t1	5	0x00000000	
t2	6	0xffffffff	
s0	7	0x00000000	
s1	8	0x00000000	

El valor se desborda, esto se programa en el ALU.

Ejercicio 5:

Es un contador en inversa, asigna el valor de 10 al registro 3 y le va restando 1 hasta que ya no tiene valores

```

1  .text
2
3      addi x3, x0, 10
4  a:
5      addi x3,x3,-1
6      bgt x3,x0, a
7
8      li a7, 10
9      ecall

```



Ejercicio 6:

Inicializa el registro 5 con el valor de 5, el 6 con el valor de 6 y el 7 con el valor de 0, después al registro 7 le suma el registro 6 y al registro 5 le resta 1 y este proceso se repite hasta que x5 es igual a 0

```
1  .text
2
3      addi x5, x0, 5
4      addi x6, x0, 6
5      addi x7, x0, 0
6  a:
7      beq x5,x0,fin
8      add x7, x7, x6
9      addi x5, x5, -1
10     b a
11  fin:
12     li a7, 10
13     ecall
```

Ejercicio 7:

```
1  .text
2
3      addi x17, x0, 10
4      addi x12, x0, 8
5      addi x7, x0, 0
6  a:
7      beq x17,x0,fin  #-- Comprueba terminacion
8      add x7, x7, x12
9      addi x17, x17, -1
10
11     #-- repetir
12
13  b:
14
15  fin:
16     li a7, 10
17     ecall
```

Bkpt	Address	Code	Basic
	0x00400000	0x00a00893	addi x17,x0,0x0000000a 3: addi x17, x0, 10
	0x00400004	0x00800613	addi x12,x0,0x00000008 4: addi x12, x0, 8
	0x00400008	0x00000393	addi x7,x0,0x00000000 5: addi x7, x0, 0
	0x0040000c	0x00088863	beq x17,x0,0x00000008 7: beq x17,x0,fin #-- Comprueba terminacion
	0x00400010	0x00e383b3	add x7,x7,x12 8: add x7, x7, x12
	0x00400014	0xffff8893	addi x17,x17,0xffff... 9: addi x17, x17, -1
	0x00400018	0xfffff06f	jal x0,0xfffff06f 13: b a
	0x0040001c	0x00a00893	addi x17,x0,0x0000000a 17: li a7, 10
	0x00400020	0x00000073	ecall 18: ecall

Es técnicamente el ejercicio anterior con otras variables.